Airline Overbooking Report

A Dynamic Programming Approach to Airline Revenue Management

Group 26:

- *Abhiroop Kumar (ak56448)*
- *Arturo Juarez (aj37739)*
- *Paolo Lopez Böttger (el33577)*
- *Thomas Garner (tg25755)*

Executive Summary

Objective: Determine the optimal daily pricing strategy and overbooking limit for a single flight over a 365 day selling timeline to **maximize expected discounted profit**.

Key Results:

- Allowing 5 maximum coach seats to be overbooked generates **$41,909.75** expected profit
- Allowing 9 maximum coach seats to be overbooked is optimal, increasing expected profit to **$42,171.35** (+0.62%)
- Allowing a no-sale day option and up to 30 overbooked coach seats raises profit to **$42,177.14** (+0.64%, +0.01%)
- Expected profit drops to **$41,864** (-$313) because heavily discounted late sales fail to offset early present value losses.
- Forward simulation of Policy A (OB=9) across 10,000 runs yields a mean discounted profit of **$42,183.65** with a standard deviation of **$906.11**
- Coach is overbooked on **84.5%** of flights; passengers are denied boarding on 73.0% of flights, with an average overbooking cost of **$1,230.61** when it occurs
- Policy B (No-Sale, OB=30) produces nearly identical results - mean profit of **$42,102.30**, confirming OB=9 already captures virtually all available value
- The primary source of profit volatility is **demand uncertainty across 365 days**, not overbooking costs; overbooking occurs on over 84% of flights but the number of affected passengers is small and the associated costs are modest relative to total revenue

Problem Context

We operate a flight with two types of available seats:

- **Coach**: 100 physical seats on the plane
- **First Class**: 20 physical seats

Over the next year until the plane leaves for takeoff, we must decide which prices to charge each day. Coach tickets can be priced at $300, $325, or $350. First class tickets at $425 or $500. Each day, at most one ticket is sold in each cabin.

The code snippet below helps us establish the final day at T=365, the capacity in each cabin, and the first scenario our team solves for.

```python
T = 365
coach_seats = 100
first_class_seats = 20
overbooking_limit = 5
max_coach_tix_sold = coach_seats + overbooking_limit
```

Our team chose to implement dynamic programming to solve this problem. In dynamic programming, we want to calculate the value function at every state our program sees, and choose the optimal decision after solving the value function for every state going backwards in time. Below are how we chose to store those values.

```python
V = np.zeros((T+1, max_coach_tix_sold+1, first_class_seats+1)) #the 366x106x21 tensor dimension from class
U = np.zeros((T+1, max_coach_tix_sold+1, first_class_seats+1), dtype=int) #stores choices from class
```

Demand Distribution

Customer purchase probabilities depend on the prices we set:

- **Coach**: 65% at $300, 45% at $325, 30% at $350
- **First Class**: 8% at $425, 4% at $500
  - Note: if first class sells out, coach purchase probability increases by 4 percentage points

The code snippet below details how our team built out these six possible choices (3 coach prices x 2 first class prices).

```python
high_coach_price = 350
high_coach_prob = 0.30
med_coach_price = 325
med_coach_prob = 0.45
low_coach_price = 300
low_coach_prob = 0.65
high_first_class_price = 500
high_first_class_prob = 0.04
low_first_class_price = 425
low_first_class_prob = 0.08

choices = [
    (low_coach_price, low_coach_prob, low_first_class_price, low_first_class_prob),
    (low_coach_price, low_coach_prob, high_first_class_price, high_first_class_prob),
    (med_coach_price, med_coach_prob, low_first_class_price, low_first_class_prob),
    (med_coach_price, med_coach_prob, high_first_class_price, high_first_class_prob),
    (high_coach_price, high_coach_prob, low_first_class_price, low_first_class_prob),
    (high_coach_price, high_coach_prob, high_first_class_price, high_first_class_prob)
]

choices
```

Python

```
[(300, 0.65, 425, 0.08),
 (300, 0.65, 500, 0.04),
 (325, 0.45, 425, 0.08),
 (325, 0.45, 500, 0.04),
 (350, 0.3, 425, 0.08),
 (350, 0.3, 500, 0.04)]
```

Overbooking and Show-Up Risk

We know that not every ticketed passenger shows up for departure:

- Coach passengers show up with 95% probability
- First class passengers show up with 97% probability

This no-show risk is the reason we've been tasked to look into overbooking the flight. If too many coach passengers arrive, though, we still have some flexibility:

- If there are available first class seats, we can **upgrade** passengers to first at a cost of $50 per upgrade
- If there are no available first class seats, we must **deny boarding** and **reimburse** passengers with **vouchers** at $425 per denied passenger

We also created objects for these show-up probabilities and overbooking costs, as well as the additional increase in sale probability mentioned earlier.

```python
coach_show_up_prob = 0.95
first_class_show_up_prob = 0.97
bump_to_fc_cost = 50
bump_off_plane_cost = 425
discount = 1/(1+0.17/365)
first_class_sold_out_boost = 0.04
```

Python

Our Method

We used **dynamic programming** to find the optimal pricing policy. The state at any point is (t, c, f) where t is the days (increasing from start t=0), c is coach tickets sold so far, and f is first class tickets sold. Starting from the day of departure and working backward, we calculated the expected profit from each state by considering all possible pricing decisions and their associated probability of sales.

Dynamic Programming Equations and Definitions

Below we define our six necessary equations, variables, and dynamics needed for dynamic programming:

1. **State:** (t, c, f) – day $t \in \{0, \ldots, 365\}$, coach tickets sold $\langle c \rangle$, FC tickets sold (f)
2. **Choice:** $p_c \in \{300, 325, 350\} \times p_f \in \{425, 500\}$ (6 combinations)
3. **Dynamics:** Stochastic - each ticket sells independently with probabilities $\theta_c(p_c)$ and $\theta_f(p_f)$; $\langle c \rangle$ and (f) increment by $s_c, s_f \in \{0, 1\}$ each day
4. **Value Function:** $V(t, c, f)$ = expected discounted profit from day (t) onward, given $\langle c \rangle$ coach and (f) FC tickets already sold
5. **Bellman Equation:**

$$V(t,c,f) = \max_{p_c, p_f} \left[\big(p_c \cdot \theta_c(p_c) + p_f \cdot \theta_f(p_f)\big) + \delta \cdot \sum_{s_c, s_f} P(s_c, s_f \mid p_c, p_f) \cdot V(t{+}1,\, c{+}s_c,\, f{+}s_f) \right]$$

6. **Terminal Condition:** $V(365, c, f) = -\mathbb{E}[\text{overbooking cost}(c, f)]$

Future profits are also discounted at an annual rate of 17%, compounded daily. This accounts for the time value of money in our revenue management decisions.

Part 1: 5 Seat Overbooking Maximum Policy

In our first analysis, **we allowed coach to be overbooked by up to 5 seats**, meaning we can sell a maximum of 105 coach tickets. The dynamic programming algorithm searches over all 6 possible price combinations (3 coach prices x 2 first class prices) to find the optimal policy at each state.

Our Method

First, we must solve for the terminal condition, since in dynamic programming we are solving the problem backwards at every possible state to determine the most optimal decisions moving forward. Therefore, we must find the values at Day 365, when the plane is taking off and in the air, for every combination of c coach tickets sold and f first class tickets sold.

Below is how our team chose to implement this.

Terminal Condition:

Finding V[365, c, f] for all possible c, f combinations

```python
for c in range(max_coach_tix_sold+1): #for every possible number of coach tickets sold
    for f in range(first_class_seats+1): #"" first class tickets sold
        cost_expected = 0
        p_coach = binom.pmf(range(c+1), c, coach_show_up_prob) #probabilities of 0, 1, ..., c coach passengers showing up
        p_first_class = binom.pmf(range(f+1), f, first_class_show_up_prob) #"" of f first class passengers showing up
        for i in range(c+1): #for every possible number of coach passengers that show up
            for j in range(f+1): #"" first class passengers that show up
                p_ij = p_coach[i] * p_first_class[j] #joint probability from class
                overbooked_coach = max(0, i - coach_seats) #overbooked coach passengers witout a coach seat
                first_class_empty_seats = max(0, first_class_seats - j) #empty first class seats
                bumped_to_first_class = min(overbooked_coach, first_class_empty_seats) #passengers bumped from coach to first class at $50/person
                bumped_off_plane = overbooked_coach - bumped_to_first_class #passengers given vouchers to new flights at $425/person
                cost = bumped_to_first_class * bump_to_fc_cost + bumped_off_plane * bump_off_plane_cost #total cost of overbooking in one scenario
                cost_expected += p_ij * cost #expected cost of overbooking across every scenario
        V[T, c, f] = -cost_expected #trying to maximize profit, minimize costs
```

Python

Next, we used the Bellman equation to loop backwards in time to find the value function at every possible state, or all combinations of t, c, and f.

```python
for t in reversed(range(T)): #loop backwards in time, starting at 364 since terminal condition covers 365
    for c in range(max_coach_tix_sold+1): #can sell 105 tix
        for f in range(first_class_seats+1): #can't overbook first class
            coach_full = (c == max_coach_tix_sold) #if c = 105, coach is full, can't sell more
            first_class_full = (f == first_class_seats) #if f = 20, first class is full, can't sell more

            if coach_full and first_class_full:
                V[t, c, f] = discount * V[t+1, c, f] #no reward today because we can't sell any more tickets, discounted future value
                U[t, c, f] = 0 #can't sell
                continue

            best_profit = -np.inf
            best_price_choice = 0

            for index, (coach_price, coach_prob, first_class_price, first_class_prob) in enumerate(choices): #for each of the 6 price combos
                prob_sell_coach = 0 if coach_full else coach_prob
                prob_sell_first_class = 0 if first_class_full else first_class_prob

                if first_class_full and not coach_full:
                    prob_sell_coach = min(1, prob_sell_coach + first_class_sold_out_boost) #increase of 4% chance of selling a coach ticket if first class is sold out

                coach_next_state = min(c+1, max_coach_tix_sold) #if we sell a coach ticket, next state is c+1, cannot > 105
                first_class_next_state = min(f+1, first_class_seats) #if we sell a first class ticket, next state is f+1, cannot > 20

                today_revenue = prob_sell_coach * coach_price + prob_sell_first_class * first_class_price #expected revenue from selling tickets today
                future_revenue = (
                    (1-prob_sell_coach) * (1-prob_sell_first_class) * V[t+1, c, f] + #sold no tickets
                    (prob_sell_coach) * (1-prob_sell_first_class) * V[t+1, coach_next_state, f] + #sold coach ticket only
                    (1-prob_sell_coach) * (prob_sell_first_class) * V[t+1, c, first_class_next_state ] + #sold first class ticket only
                    (prob_sell_coach) * (prob_sell_first_class) * V[t+1, coach_next_state, first_class_next_state] #sold both tickets
                )

                profit = today_revenue + discount * future_revenue

                if profit > best_profit:
                    best_profit = profit
                    best_price_choice = index + 1

            V[t, c, f] = best_profit
            U[t, c, f] = best_price_choice
```

Python

Now, every state variable combination is stored in the V tensor, while every optimal decision is in the U object. With these, we can understand the expected discounted profit today (Day 0) of making decisions optimally according to the value function:

Results

Expected profit: $41,909.75

The figure below shows the optimal pricing policy as a heatmap. The x-axis represents the number of days tickets have been live (starting at t=0, 365 days until takeoff), and the y-axis shows coach tickets already sold. Each cell is colored according to the price combination chosen.

To simplify the visual as well, we reduced our three-dimensional state space (t, c, f) to just (t, c) and kept first class tickets sold at 0. This is the most effective way we understood the pricing policies our dynamic program recommended, since we prioritize coach tickets sold while balancing the overbooking limit.

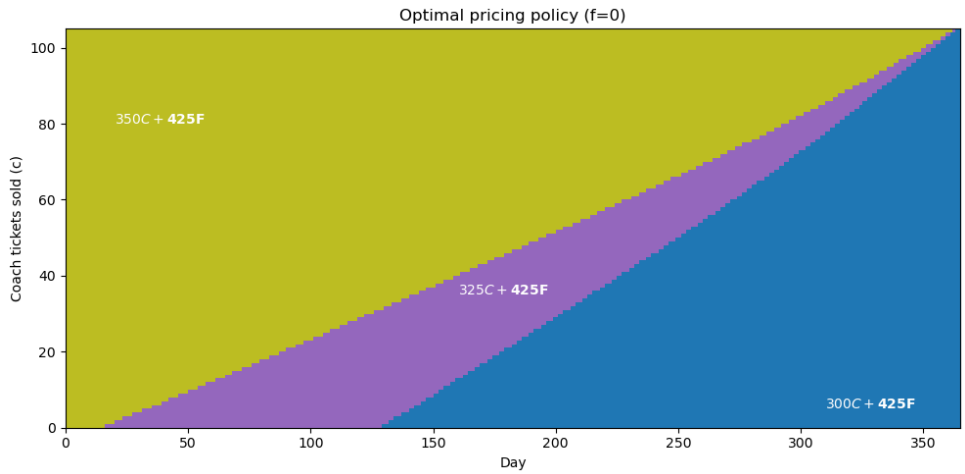

Figure 1: Optimal pricing policy heatmap (5-seat overbooking cap). Assuming f=0

The policy shows distinct pricing zones. Early in the selling period (many days remaining), when few tickets have been sold, the airline charges a high price for coach ($350) and a low price for first class ($425). As departure approaches and as more tickets sell, coach prices drop to the medium price point ($325) shown in the purple shaded region. Eventually, if coach tickets still haven't sold and the flight is close to departure, we recommend setting the lowest price for coach ($300) to encourage those fringe customers who are price-sensitive. Early buyers are less price-sensitive, while last-minute inventory risk requires lower prices to ensure our flight seats are filled.

Figure 2 provides an additional view of the solution. To understand the discounted value of expected profits, our team visualized the value function over time. Considering the value function sets prices optimally every day, it is clearly more valuable for seats to sell sooner rather than later, as the value function tanks around day 200.

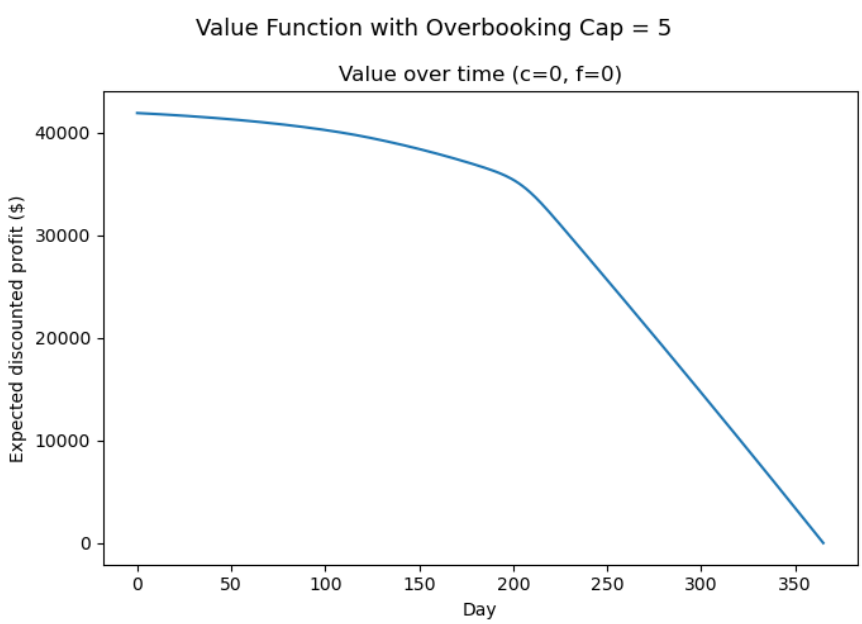

Figure 2: Expected profit over time

Most of our profit comes from these early sales, where the value function over time is somewhat linear, which tells our team that there is a limited time to maximize these profits, incentivizing our company to charge the highest prices.

In fact, if we were to look at how expected profits change at different snapshots in time, we can see that the value of each additional coach ticket sold decreases as takeoff approaches. It is important that we set prices optimally before we have too much inventory to sell and must cut costs. Early in the selling period (100 days in), the value function is essentially linear, where each unsold seat has about equal future value. Closer to takeoff (200 days in), expected profits begin to flatten, showing an unequal distribution of coach seats sold. Then, our team saw the growing cost of overbooking risk 300 days in, where there is simply less value in selling a coach ticket when there is bulk inventory.

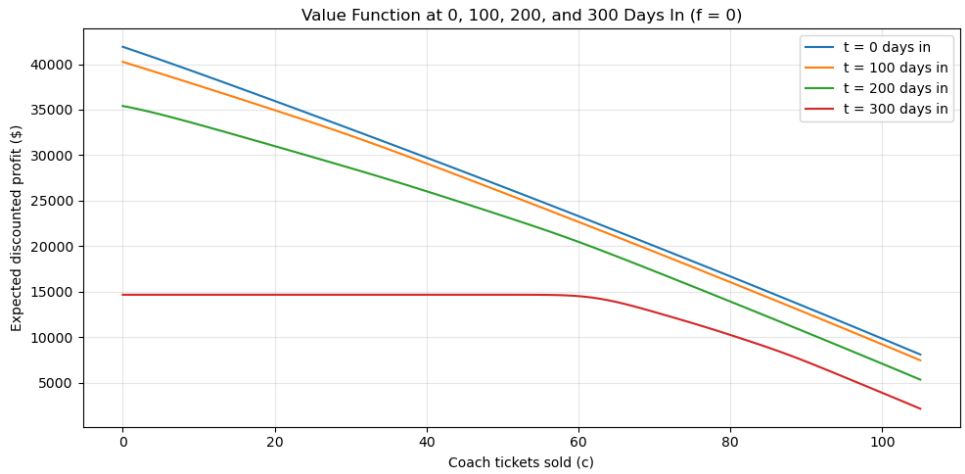

Figure 3: Expected Profit at 3 Time Snapshots

It is clear that our team must begin selling coach tickets before too much time passes. If we find ourselves at Day 300 with no coach tickets sold yet, we are already at a loss and can only make up a fraction of the potential expected discounted profit. This figure captures the value add of pricing optimally with our dynamic programming.

Part 2: Optimizing the Overbooking Limit

Next, we tested different overbooking limits to find which maximizes expected profit. Instead of fixing the coach capacity at 105 tickets (5 extra seats), we **varied the limit from 5 to 20 extra seats** and re-ran the dynamic program for each setting.

Our Method

For this, our team decided to define a Python function that we could then loop over these combinations of extra seats to overbook. Below shows how we consolidated the objects, terminal condition, and Bellman equation from Part 1.

```python
simulate_policy(overbooking_limit):
max_coach_tix_sold = coach_seats + overbooking_limit

V = np.zeros((T+1, max_coach_tix_sold+1, first_class_seats+1)) #the 366x106x21 tensor dimension from class
U = np.zeros((T+1, max_coach_tix_sold+1, first_class_seats+1), dtype=int) #stores choices from class

#terminal condition:
for c in range(max_coach_tix_sold+1): #for every possible number of coach tickets sold
    for f in range(first_class_seats+1): #"" first class tickets sold
        cost_expected = 0
        p_coach = binom.pmf(range(c+1), c, coach_show_up_prob) #probabilities of 0, 1, ..., c coach passengers showing up
        p_first_class = binom.pmf(range(f+1), f, first_class_show_up_prob) #"" of f first class passengers showing up
        for i in range(c+1): #for every possible number of coach passengers that show up
            for j in range(f+1): #"" first class passengers that show up
                p_ij = p_coach[i] * p_first_class[j] #joint probability from class
                overbooked_coach = max(0, i - coach_seats) #overbooked coach passengers witout a coach seat
                first_class_empty_seats = max(0, first_class_seats - j) #empty first class seats
                bumped_to_first_class = min(overbooked_coach, first_class_empty_seats) #passengers bumped from coach to first class at $50/person
                bumped_off_plane = overbooked_coach - bumped_to_first_class #passengers given vouchers to new flights at $425/person
                cost = bumped_to_first_class * bump_to_fc_cost + bumped_off_plane * bump_off_plane_cost #total cost of overbooking in one scenario
                cost_expected += p_ij * cost #expected cost of overbooking across every scenario
        V[T, c, f] = -cost_expected #trying to maximize profit, minimize costs

#bellman equation
for t in reversed(range(T)): #loop backwards in time, starting at 364 since terminal condition covers 365
    for c in range(max_coach_tix_sold+1): #can sell 105 tix
        for f in range(first_class_seats+1): #can't overbook first class
            coach_full = (c == max_coach_tix_sold) #if c = 105, coach is full, can't sell more
            first_class_full = (f == first_class_seats) #if f = 20, first class is full, can't sell more

            if coach_full and first_class_full:
                V[t, c, f] = discount * V[t+1, c, f] #no reward today because we can't sell any more tickets, discounted future value
                U[t, c, f] = 0 #can't sell
                continue

            best_profit = -np.inf
            best_price_choice = 0

            for index, (coach_price, coach_prob, first_class_price, first_class_prob) in enumerate(choices): #for each of the 6 price combos
                prob_sell_coach = 0 if coach_full else coach_prob
                prob_sell_first_class = 0 if first_class_full else first_class_prob

                if first_class_full and not coach_full:
                    prob_sell_coach = min(1, prob_sell_coach + first_class_sold_out_boost) #increase of 4% chance of selling a coach ticket if first class is sold out

                coach_next_state = min(c+1, max_coach_tix_sold) #if we sell a coach ticket, next state is c+1, cannot > 105
                first_class_next_state = min(f+1, first_class_seats) #if we sell a first class ticket, next state is f+1, cannot > 20

                today_revenue = prob_sell_coach * coach_price + prob_sell_first_class * first_class_price #expected revenue from selling tickets today
                future_revenue = (
                    (1-prob_sell_coach) * (1-prob_sell_first_class) * V[t+1, c, f] + #sold no tickets
                    (prob_sell_coach) * (1-prob_sell_first_class) * V[t+1, coach_next_state, f] + #sold coach ticket only
                    (1-prob_sell_coach) * (prob_sell_first_class) * V[t+1, c, first_class_next_state ] + #sold first class ticket only
                    (prob_sell_coach) * (prob_sell_first_class) * V[t+1, coach_next_state, first_class_next_state] #sold both tickets
                )

                profit = today_revenue + discount * future_revenue

                if profit > best_profit:
                    best_profit = profit
                    best_price_choice = index + 1

            V[t, c, f] = best_profit
            U[t, c, f] = best_price_choice

return V, U, round(V[0, 0, 0], 2)
```

Python

After confirming our answer from Part 1 using this defined function, we ran a for loop to answer the Part 2 situation of searching for the optimal overbooking limit.

```python
for i in range(5, 21): #from 5 to 20, the range of overbooking limits to check
    V, U, expected_profit = simulate_policy(i) #grab the expected profit for each overbooking limit
    print(f"Overbooking limit: {i}, Expected profit: {expected_profit}")
```

Python

```
Overbooking limit: 5, Expected profit: 41909.75
Overbooking limit: 6, Expected profit: 42039.02
Overbooking limit: 7, Expected profit: 42117.95
Overbooking limit: 8, Expected profit: 42157.77
Overbooking limit: 9, Expected profit: 42171.35
Overbooking limit: 10, Expected profit: 42169.24
Overbooking limit: 11, Expected profit: 42158.86
Overbooking limit: 12, Expected profit: 42144.91
Overbooking limit: 13, Expected profit: 42130.13
Overbooking limit: 14, Expected profit: 42115.88
Overbooking limit: 15, Expected profit: 42102.78
Overbooking limit: 16, Expected profit: 42091.04
Overbooking limit: 17, Expected profit: 42080.72
Overbooking limit: 18, Expected profit: 42071.78
Overbooking limit: 19, Expected profit: 42064.15
Overbooking limit: 20, Expected profit: 42057.69
```

Results

Optimal Overbooking Setting: 9 extra coach seats (109 total capacity)

Expected Profit: $42,171.35 (+0.62%)

Figure 4 shows the expected profit curve across these different overbooking limits in our search.

Figure 4: Expected Profit vs Overbooking Limit

Profits initially rise fast as we allow more overbooking because we can sell more tickets and revenue increases faster than overbooking costs. However, beyond 9 extra seats, the cost of denied boardings and upgrades grows faster than revenue. The profit curve peaks at 9 seats and declines afterwards, confirming that our optimal policy balances revenue opportunity and risk.

Figure 5 compares the two pricing policies thus far: 5 and 9 seats overbooking maximum in coach.

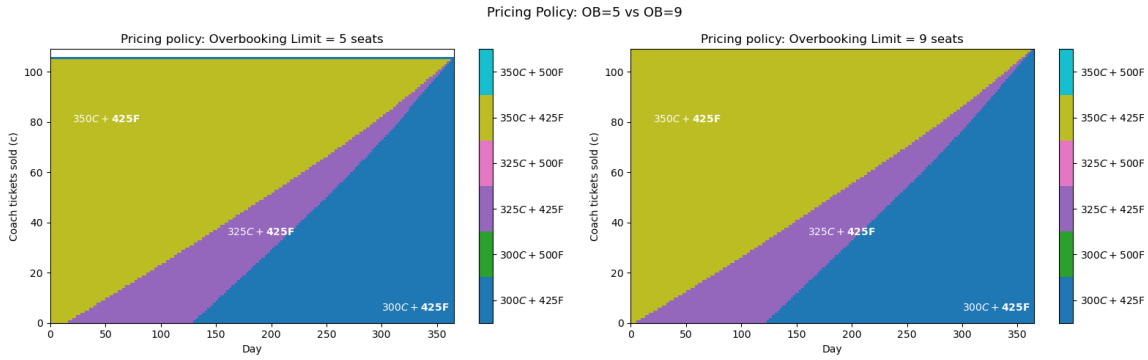

Figure 5: Side-by-side Policy Heatmaps for 5-seat and 9-seat Overbooking Limits

The two policies look similar because both face the same demand structure. The main difference is that the 9-seat policy can sell more tickets before hitting capacity constraints (as shown in the top row).

Since Figure 5 is hard to tell the differences between the two policies in most of the states, Figure 6 was created to highlight where exactly the two policies differ by subtracting their price choices.

Yellow regions represent identical pricing decisions. Red diagonal lines show where the pricing policies are different. These boundaries represent states where the 9-seat policy chooses a slightly lower coach price (moving from $350 to $325) to fill remaining capacity without excessive overbooking risk. The 9-seat policy is more aggressive in pursuing revenue because the policy has more room before hitting

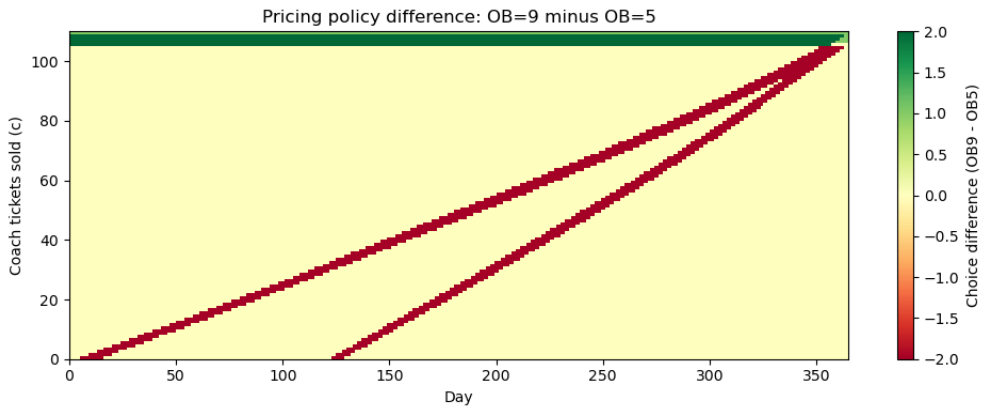

Figure 6: Difference Heatmap (9-seat Policy Minus 5-seat Policy)

any overbooking territory. The green strip at the top also represents the space unavailable for the 5-seat capacity and the extra decisions our dynamic program accounted for (mostly $350 for coach tickets).

Part 3: Adding a No-Sale Option

In the previous parts, we explored how overbooking coach seats could lead to an increase in revenue, and then we pivoted to finding the optimal overbooking number for coach seats. Within these approaches, we were required to sell coach tickets every day. Now, we will introduce **a fourth coach option in which we sell nothing**. Thus, giving our pricing team flexibility to withhold inventory when the overbooking risk is too high. We will set a soft ceiling of 130 coach tickets as the maximum we would ever sell.

Our Method

Now that we are introducing a 7th and 8th option into our choice matrix and increasing the overbooking limit to 30, our team made the following edits to our static objects, V and U tensors, and choice matrix.

```python
T = 365
coach_seats = 100
first_class_seats = 20
overbooking_limit = 30
max_coach_tix_sold = coach_seats + overbooking_limit
max_coach_tix_sold #check
```

Python

```
130
```

```python
V = np.zeros((T+1, max_coach_tix_sold+1, first_class_seats+1)) #the 366x131x21 tensor dimension now being able to book up to 30 extra coach seats
U = np.zeros((T+1, max_coach_tix_sold+1, first_class_seats+1), dtype=int) #stores choices from class
```

Python

```python
V.shape
```

Python

```
(366, 131, 21)
```

```python
high_coach_price = 350
high_coach_prob = 0.30
med_coach_price = 325
med_coach_prob = 0.45
low_coach_price = 300
low_coach_prob = 0.65
no_coach_price = 0
no_coach_prob = 0
high_first_class_price = 500
high_first_class_prob = 0.04
low_first_class_price = 425
low_first_class_prob = 0.08

choices = [
    (low_coach_price, low_coach_prob, low_first_class_price, low_first_class_prob),
    (low_coach_price, low_coach_prob, high_first_class_price, high_first_class_prob),
    (med_coach_price, med_coach_prob, low_first_class_price, low_first_class_prob),
    (med_coach_price, med_coach_prob, high_first_class_price, high_first_class_prob),
    (high_coach_price, high_coach_prob, low_first_class_price, low_first_class_prob),
    (high_coach_price, high_coach_prob, high_first_class_price, high_first_class_prob),
    (no_coach_price, no_coach_prob, low_first_class_price, low_first_class_prob),
    (no_coach_price, no_coach_prob, high_first_class_price, high_first_class_prob)
]

choices
```

Python

```
[(300, 0.65, 425, 0.08),
 (300, 0.65, 500, 0.04),
 (325, 0.45, 425, 0.08),
 (325, 0.45, 500, 0.04),
 (350, 0.3, 425, 0.08),
 (350, 0.3, 500, 0.04),
 (0, 0, 425, 0.08),
 (0, 0, 500, 0.04)]
```

```python
coach_show_up_prob = 0.95
first_class_show_up_prob = 0.97
bump_to_fc_cost = 50
bump_off_plane_cost = 425
discount = 1/(1+0.17/365)
first_class_sold_out_boost = 0.04
```

Python

Since we were running this dynamic program for only one overbooking limit, we just copy and pasted the terminal condition and Bellman equation calculations from Part 1. Then we ran the program for this new scenario.

Results

Expected Profit: $42,177.14 (+0.64% from OB=5, +0.01% from OB=9)

Figure 7 shows where the no-sale option is actually chosen, depicted in green. The additional no-sale option is chosen in 7% of states. As shown in Figure 7, the no-sale option (represented in green) occurs in the upper/upper-right corner of the state space. In other words, the no-sale option is mainly chosen when many coach tickets have already been sold, and never before the 50th day of tickets being live. In these instances, one more ticket being pushed would cause overbooking costs to exceed the revenue gained. Therefore, the airline withholds inventory in these scenario states.

Once we've added the no-sale option to the overall scenarios from before, we have 8 possible choices (3 coach prices x 2 FC prices + 1 coach no-sale x 2 FC prices). As shown in Figure 8, the overall structure remains the same, with high prices early and low prices late. With the addition of the no-sale zones at the top, in which the airlines do not offer coach tickets when there are already many coach tickets sold.

Now, when we compare the state graphs for the optimal 9-overbooking strategy and the additional no-sale option, they appear to have the same structure (Figure 9). The key difference is that the no-sale option offers more than 105 coach tickets, with the caveat that there are instances when it is better not to offer coach tickets for sale.

Figure 10 shows a clearer overlap comparison, highlighting the differences between the no-sale option and the optimal overbooking of 9 coach seats.

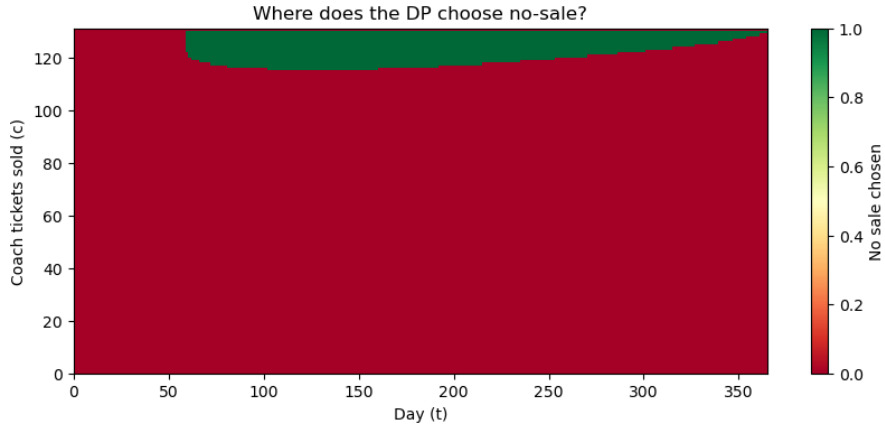

Figure 7. States where the no-sale option is chosen

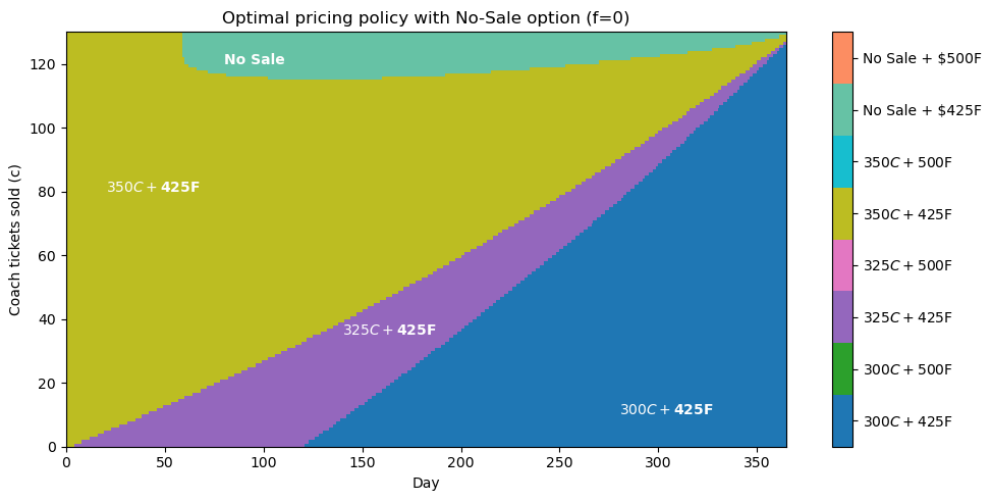

Figure 8. Pricing policy heatmap with no-sale option included

Figure 9. Side-by-side comparison between OB = 9 and no-sale option

The yellow regions indicate identical decisions, in which the pricing strategy is the same for the overbooking 9 and the no-sale strategy. The differences between the strategies lie in the states at which the pricing strategy changes. These differences are depicted by the orange and red regions, with the orange region indicating when the no-sale strategy offers a price tier lower than the overbooking 9-coach-seat strategy. The red region indicates when the price for the no-sale strategy drops to 2 stages below.

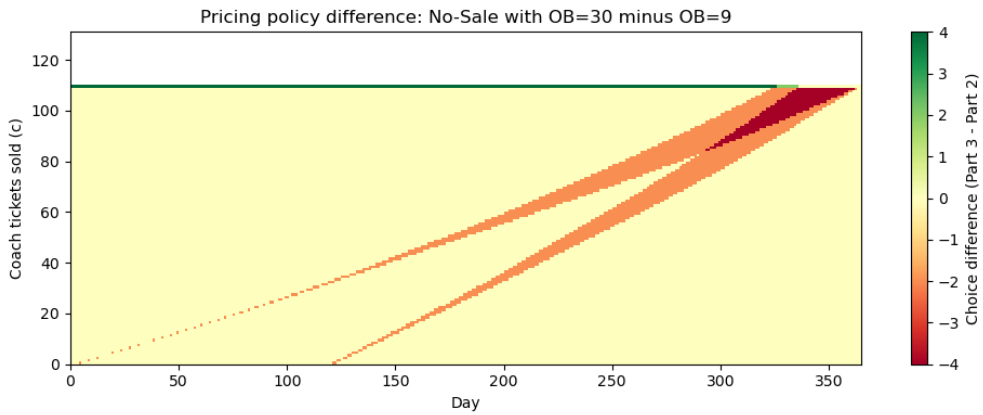

Figure 10. Difference Heatmap between no-sale option and OB = 9

So far, we have explored three strategies: OB = 5, OB = 9, and the no-sale option. The first option, in which the coach's overbooking cap is 5, is too conservative since it leaves money on the table. When that overbooking cap is raised to the optimal cap of 9, we can capture most of the available upside. On top of this, the no-sale option provides a slight increase in expected profit by allowing us to raise the cap higher, while giving the pricing team the option not to offer coach tickets in extreme situations where the cost of overbooking would cut into profits.

Overall, Figure 11 shows the expected profits across the three strategies, with a significant improvement from the first to the second strategy, followed by a slight improvement when incorporating the third strategy (no-sale option).

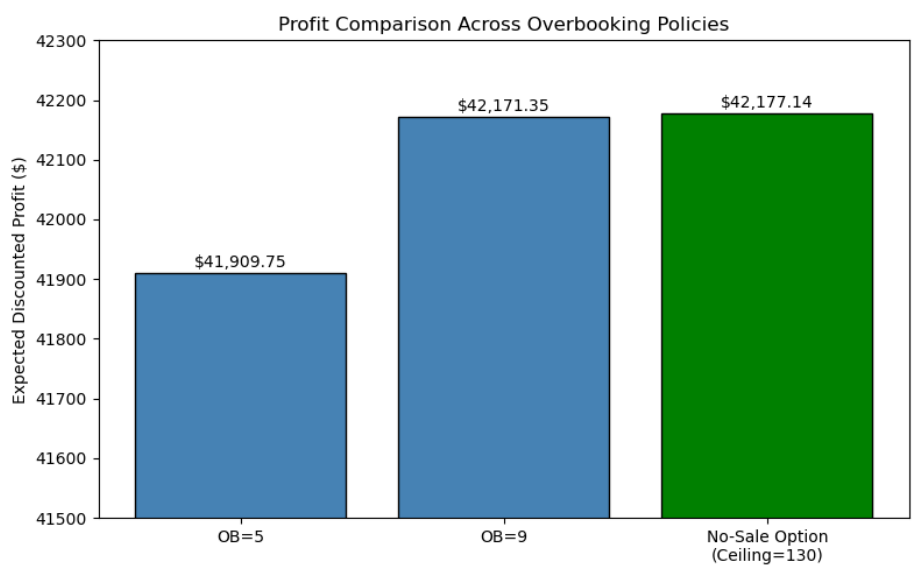

Figure 11.Expected profit comparison across OB = 5, 9, and the no-sale option

Part 4: Sensitivity Analysis

Previously, we assumed that the probabilities for sales were fixed: 65%, 45%, and 30% for coach, and 8% and 4% for first class. However, in the real world, these probabilities would not be fixed and would carry uncertainty.

Our Method

To assess how robust our optimal policy is to demand estimation errors, we fixed the overbooking limit at OB=9 and varied each of the five sale probabilities, one at a time, in 1pp increments from −5pp to +5pp, holding the other four at their baseline values. This required re-running the full dynamic program for each perturbed probability.

```
def simulate_policy_with_probs(overbooking_limit,
                               lcp=0.65, mcp=0.45, hcp=0.30,
                               lfcp=0.08, hfcp=0.04):
    #same DP as simulate_policy(), but accepts custom sale probabilities for sensitivity analysis
    max_ctix = coach_seats + overbooking_limit

    local_choices = [ #rebuild choices list with the supplied probabilities
        (low_coach_price,  lcp,  low_first_class_price,  lfcp),
        (low_coach_price,  lcp,  high_first_class_price, hfcp),
        (med_coach_price,  mcp,  low_first_class_price,  lfcp),
        (med_coach_price,  mcp,  high_first_class_price, hfcp),
        (high_coach_price, hcp,  low_first_class_price,  lfcp),
        (high_coach_price, hcp,  high_first_class_price, hfcp),
    ]

    V = np.zeros((T+1, max_ctix+1, first_class_seats+1))
    U = np.zeros((T+1, max_ctix+1, first_class_seats+1), dtype=int)

    #terminal condition:
    for c in range(max_ctix+1): #for every possible number of coach tickets sold
        for f in range(first_class_seats+1): #"" first class tickets sold
            cost_expected = 0
            p_coach       = binom.pmf(range(c+1), c, coach_show_up_prob) #probabilities of 0, 1, ..., c coach passengers showing up
            p_first_class = binom.pmf(range(f+1), f, first_class_show_up_prob) #"" of f first class passengers showing up
            for i in range(c+1): #for every possible number of coach passengers that show up
                for j in range(f+1): #"" first class passengers that show up
                    p_ij             = p_coach[i] * p_first_class[j] #joint probability
                    overbooked_coach = max(0, i - coach_seats) #overbooked coach passengers without a coach seat
                    fc_empty_seats   = max(0, first_class_seats - j) #empty first class seats
                    bumped_to_fc     = min(overbooked_coach, fc_empty_seats) #passengers bumped to first class at $50/person
                    bumped_off       = overbooked_coach - bumped_to_fc #passengers given vouchers at $425/person
                    cost             = bumped_to_fc * bump_to_fc_cost + bumped_off * bump_off_plane_cost #total cost in one scenario
                    cost_expected   += p_ij * cost #expected cost across all scenarios
            V[T, c, f] = -cost_expected #trying to maximize profit, minimize costs

    #bellman equation
    for t in reversed(range(T)): #loop backwards in time, starting at 364 since terminal condition covers 365
        for c in range(max_ctix+1): #can sell up to max_ctix tickets
            for f in range(first_class_seats+1): #can't overbook first class
                coach_full       = (c == max_ctix) #if c = max_ctix, coach is full, can't sell more
                first_class_full = (f == first_class_seats) #if f = 20, first class is full, can't sell more

                if coach_full and first_class_full:
                    V[t, c, f] = discount * V[t+1, c, f] #no reward today, discounted future value
                    U[t, c, f] = 0 #can't sell
                    continue

                best_profit = -np.inf
                best_choice = 0

                for idx, (cp, cpp, fp, fpp) in enumerate(local_choices): #for each of the 6 price combos
                    p_c = 0 if coach_full else cpp
                    p_f = 0 if first_class_full else fpp

                    if first_class_full and not coach_full:
                        p_c = min(1, p_c + first_class_sold_out_boost) #increase of 4% chance of selling a coach ticket if first class is sold out

                    cn = min(c+1, max_ctix) #if we sell a coach ticket, next state is c+1, cannot exceed max_ctix
                    fn = min(f+1, first_class_seats) #if we sell a first class ticket, next state is f+1, cannot exceed 20

                    today_rev  = p_c * cp + p_f * fp #expected revenue from selling tickets today
                    future_rev = (
                        (1-p_c)*(1-p_f) * V[t+1, c,  f ] + #sold no tickets
                        (  p_c)*(1-p_f) * V[t+1, cn, f ] + #sold coach ticket only
                        (1-p_c)*(  p_f) * V[t+1, c,  fn] + #sold first class ticket only
                        (  p_c)*(  p_f) * V[t+1, cn, fn]   #sold both tickets
                    )
                    profit = today_rev + discount * future_rev

                    if profit > best_profit:
                        best_profit = profit
                        best_choice = idx + 1

                V[t, c, f] = best_profit
                U[t, c, f] = best_choice

    return V, U, round(V[0, 0, 0], 2)
```

```python
#perturb each probability by -5 to +5 percentage points in 1pp increments
deltas = np.round(np.arange(-0.05, 0.06, 0.01), 3) #[-0.05, -0.04, ..., +0.05]

prob_params = { #human-readable label mapped to (kwarg name, baseline value)
    '$300 coach (65%)': ('lcp',  0.65),
    '$325 coach (45%)': ('mcp',  0.45),
    '$350 coach (30%)': ('hcp',  0.30),
    '$425 FC   ( 8%)': ('lfcp', 0.08),
    '$500 FC   ( 4%)': ('hfcp', 0.04),
}

sensitivity_results = {} #label mapped to list of profits, one per delta

for label, (kwarg, base_val) in prob_params.items():
    profits = []
    for delta in deltas:
        new_val = float(np.clip(base_val + delta, 0.0, 1.0)) #clip to valid probability range
        kwargs = dict(lcp=0.65, mcp=0.45, hcp=0.30, lfcp=0.08, hfcp=0.04) #start from baseline
        kwargs[kwarg] = new_val #override only the one being perturbed
        _, _, profit = simulate_policy_with_probs(9, **kwargs)
        profits.append(profit)
    sensitivity_results[label] = profits
    print(f"  {label}  |  range: ${min(profits):,.0f} to ${max(profits):,.0f}")

print(f"\nBaseline profit: ${baseline_profit:,.2f}")
```

```
  $300 coach (65%)  |  range: $42,170 to $42,173
  $325 coach (45%)  |  range: $42,143 to $42,214
  $350 coach (30%)  |  range: $41,602 to $42,726
  $425 FC   ( 8%)  |  range: $41,179 to $42,489
  $500 FC   ( 4%)  |  range: $41,909 to $43,422

Baseline profit: $42,171.35
```

Results

First, we need to establish the baseline expected profit for the 9-seat overbooking cap strategy: $42,171.35. Figure 12 shows the sensitivity of expected profit to each probability, including the profit curves, the profit change from the baseline, a tornado chart ranking the total impact, and a ranked slope chart showing the profit change per percentage point.

First-class demand probabilities are the most influential inputs. Changing the $500 first-class ticket sale probability (baseline 4%), from -5pp to +5pp in 1pp increments, moves expected profit from $41,909 to $43,422, a total range of $1,513. This is about $151 per percentage point. The $425 FC probability is close behind at $131/pp. Despite there being only 20 physical first-class seats, the high ticket prices mean each additional expected sale contributes heavily to total revenue.

Among coach prices, the $350 tier matters the most. Changing the $350 coach probability (baseline: 30%) in 1pp increments results in a range of $1,124, comparable to first class. As noted previously, the state graph showed that the $350 coach ticket is used most often early in the selling window, so its probability drives the majority of early revenue. The other two coach tiers ($300 and $325) have a very small effect on expected profit. Changing the $325 coach probability yields only a $71 range, and the $300 coach probability is virtually irrelevant, less than $4 total range, because the dynamic program uses that price only near departure, when marginal seat value is already low.

Therefore, the demand estimation effort should focus on first-class demand and the $350 coach tier. A 1pp error in the $500 FC probability costs ~$151 in expected profit, while a 1pp error in the $300 coach probability costs less than $0.50.

Part 4: Sensitivity of Expected Profit to Sale Probabilities (OB = 9)

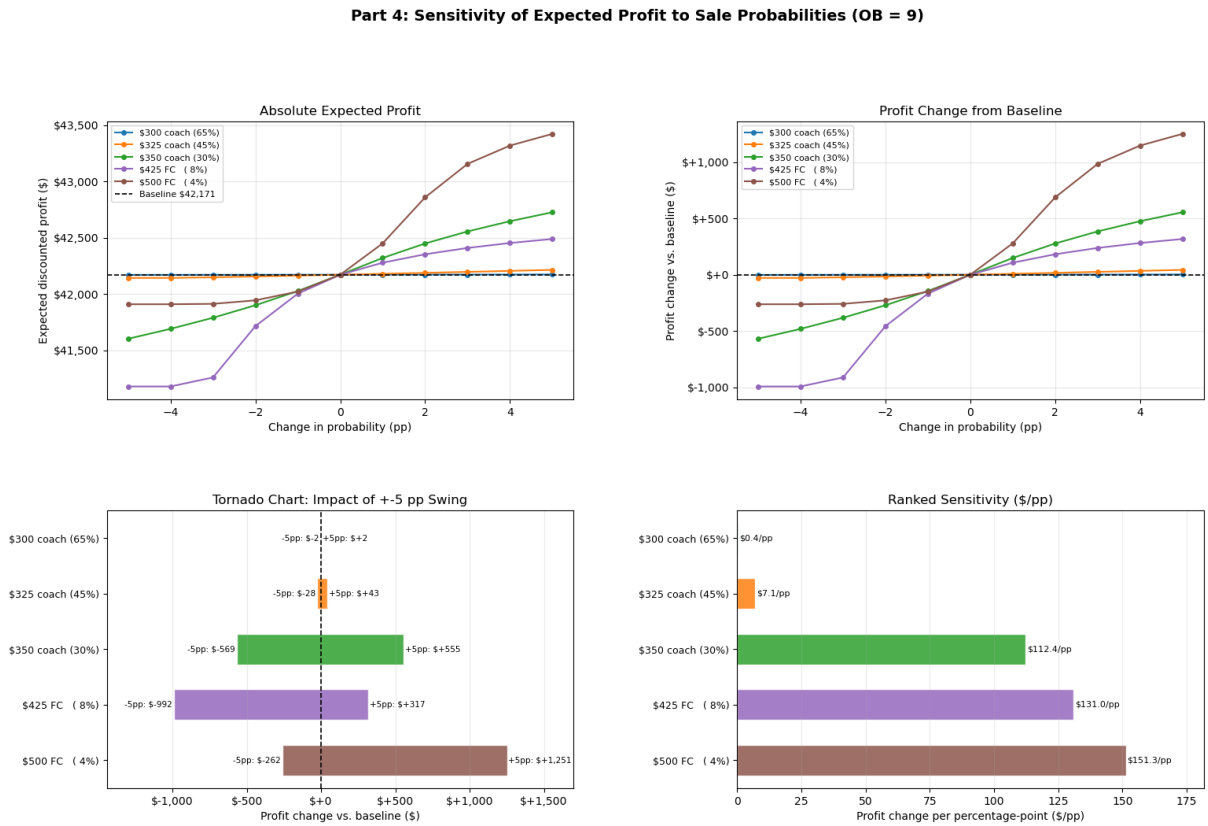

Figure 12. Sensitivity of Expected Profit to Sale Probabilities (OB = 9)

Part 5: Seasonality - Time-Varying Demand

Our Method

We extend the Part 3 DP to incorporate time-varying demand. On each selling day t (where t = 0 is 365 days before departure and t = 364 is the day before departure), every sale probability is multiplied by a seasonal factor:

$$\text{factor}(t) = 0.75 + \frac{t}{730}$$

This produces a smooth, linear increase in demand as departure approaches. At t = 0, the factor is 0.75, so demand is 25% below the flat baseline. At t $\approx$ 182, the factor reaches 1.00 and matches the baseline. At t = 364, the factor is 1.25, so demand is 25% above baseline.

In the Bellman equation, each day's choice probabilities are scaled by this factor before computing expected revenues. All other parameters remain identical to Part 3: overbooking limit OB = 30, the no-sale option, the same discount factor, and the same bump costs. The key code modification is:

- Seasonal Demand Multiplier = 0.75 + t / 730

- Low coach price probability ($300) = min(1.0, low_coach_prob * demand_mult)
- Medium coach price probability ($325) = min(1.0, med_coach_prob * demand_mult)
- High coach price probability ($350) = min(1.0, high_coach_prob * demand_mult)
- Low first class price probability ($425) = min(1.0, low_fc_prob * demand_mult)
- High first class price probability ($500) = min(1.0, high_fc_prob * demand_mult)

This was implemented in the code in the following structure:

```
# Bellman equation with time-varying probabilities
for t in reversed(range(T)):
    season_factor = min(1.0 / max(p for _, p, _, _ in choices if p > 0),
                        0.75 + t / 730)
    # Safer: compute factor independently; cap each prob at 1
    factor = 0.75 + t / 730

    for c in range(max_c + 1):
        for f in range(first_class_seats + 1):
            c_full  = (c == max_c)
            fc_full = (f == first_class_seats)
            if c_full and fc_full:
                V[t, c, f] = discount * V[t + 1, c, f]
                U[t, c, f] = 0
                continue
            best_profit = -np.inf
            best_choice = 0
            for idx, (cp, cpr, fp, fpr) in enumerate(choices):
                # Scale probabilities by seasonal factor, cap at 1
                ps_c  = 0 if c_full  else min(1.0, cpr * factor)
                ps_fc = 0 if fc_full else min(1.0, fpr * factor)
                if fc_full and not c_full and cpr > 0:
                    # Boost applies after scaling, still cap at 1
                    ps_c = min(1.0, ps_c + fc_boost)
                cn = min(c + 1, max_c)
                fn = min(f + 1, first_class_seats)
                today = ps_c * cp + ps_fc * fp
                future = (
                    (1 - ps_c) * (1 - ps_fc) * V[t+1, c,  f ] +
                    ps_c       * (1 - ps_fc) * V[t+1, cn, f ] +
                    (1 - ps_c) * ps_fc        * V[t+1, c,  fn] +
                    ps_c       * ps_fc         * V[t+1, cn, fn]
                )
                profit = today + discount * future
                if profit > best_profit:
                    best_profit = profit
                    best_choice = idx + 1
            V[t, c, f] = best_profit
            U[t, c, f] = best_choice
return round(V[0, 0, 0], 2), V, U
```

Results

Introducing time varying demand reduces expected discounted profit from $42,177 to $41,864, a decrease of 313 dollars compared to the problem 3. This result highlights a critical interaction between demand patterns and time value of money.

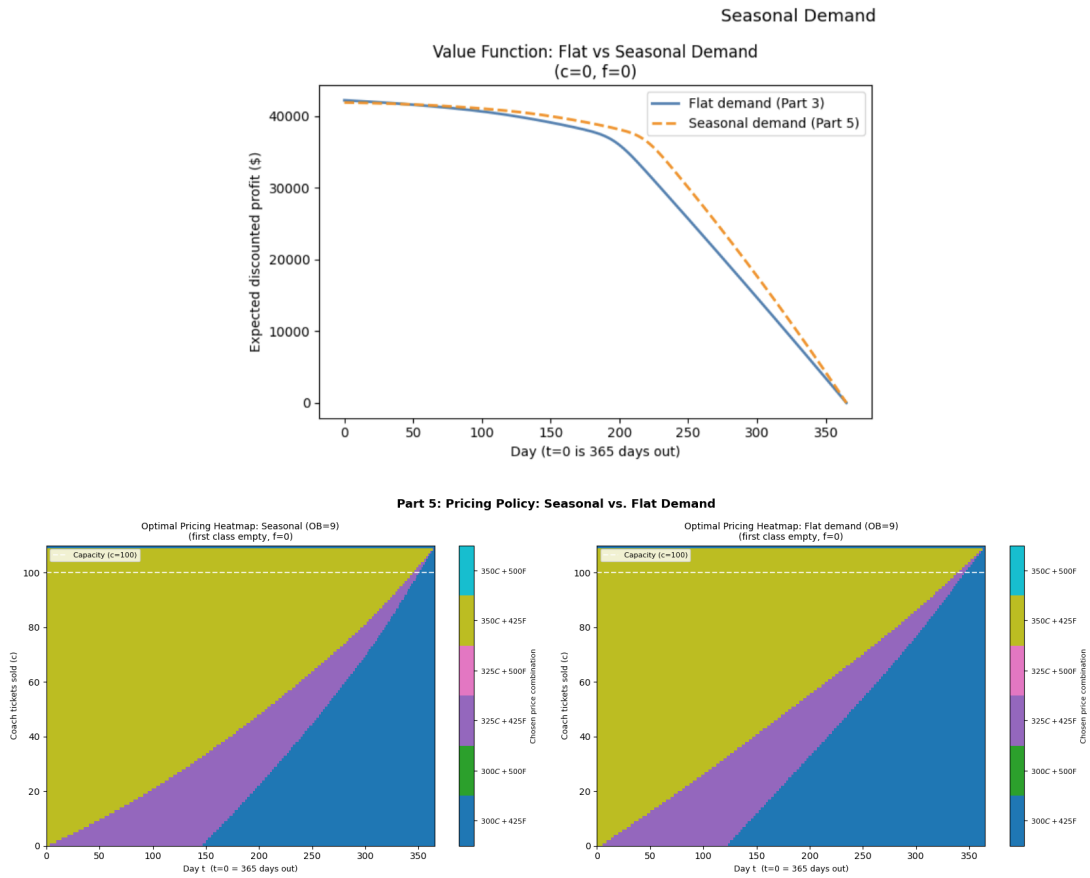

Figure 13.Comparison of Value Functions and State Spaces Between Flat and Seasonal Demand Assumptions

With the seasonal demand assumption, our team found some differences when it comes to the value function and state spaces at different days (t). Below are some findings:

- Value erosion: during the first 182 days approximately, the demand is suppressed, because these early revenues are subject to less discounting , their reduction significantly impacts the total present value.
- Discounted gains: Even though the demand surges in the second half of the window, these earnings occur closer to departure and as a consequence they are heavily discounted.
- Policy adaptation: The optimal pricing policy becomes more protective. Under seasonality, the system switches to higher price points earlier in the window to save seats for the late demand surge.

Part 6: Forward Simulation

Up to this point, we solved the pricing problem backwards using dynamic programming, starting from departure and working back to Day 0. This gives us the theoretically optimal expected profit. But theory and practice can differ: real flights experience randomness in who actually buys a ticket each day and who shows up at the gate.

Our Method

To validate our policies and understand the distribution of outcomes (not just the expected value), we simulated each policy forward 10,000 times. Each simulation runs through all 365 days, making a pricing decision each day according to the optimal policy stored in U, randomly determining whether a sale occurs based on that day's probabilities, and then on departure day, drawing a binomial random variable to determine how many ticketed passengers actually show up.

We compared two policies head-to-head:

- **Policy A - Hard Cap OB=9** (best from Part 2): sell at most 109 coach tickets, no no-sale option
- **Policy B - Flexible No-Sale OB=30** (Part 3): can choose to withhold coach tickets on any day, ceiling of 130

For each of the 10,000 simulations, we tracked: discounted profit, coach and first class tickets sold, overbooking events, and passengers denied boarding.

```python
def forward_simulate(U, choices_list, max_coach_tix, n_sims=10_000, seed=42):
    #simulate the optimal policy forward n_sims times and collect operational metrics
    rng = np.random.default_rng(seed)

    profits    = np.zeros(n_sims) #discounted net profit per simulation
    ob_costs   = np.zeros(n_sims) #overbooking cost per simulation (undiscounted)
    coach_sold = np.zeros(n_sims, dtype=int)
    fc_sold    = np.zeros(n_sims, dtype=int)
    overbooked = np.zeros(n_sims, dtype=int) #coach passengers without a seat
    kicked     = np.zeros(n_sims, dtype=int) #passengers bumped off the plane entirely

    for sim in range(n_sims):
        c, f         = 0, 0 #state: tickets sold so far
        disc_revenue = 0.0
        disc_t       = 1.0 #discount factor starts at 1 for t=0

        for t in range(T):
            choice_idx = U[t, c, f]

            if choice_idx == 0: #both cabin classes full, no tickets to sell
                disc_t *= discount
                continue

            cp, cpp, fp, fpp = choices_list[choice_idx - 1]
```

```python
            p_c = 0.0 if c >= max_coach_tix    else cpp #no coach sale if at cap
            p_f = 0.0 if f >= first_class_seats else fpp #no FC sale if sold out

            if f >= first_class_seats and c < max_coach_tix:
                p_c = min(1.0, p_c + first_class_sold_out_boost) #4% boost to coach demand when FC is sold out

            sold_c = bool(p_c > 0 and rng.random() < p_c) #simulate whether coach ticket sold
            sold_f = bool(p_f > 0 and rng.random() < p_f) #simulate whether FC ticket sold

            disc_revenue += disc_t * (int(sold_c) * cp + int(sold_f) * fp)
            c += int(sold_c)
            f += int(sold_f)
            disc_t *= discount

        coach_sold[sim] = c
        fc_sold[sim]    = f

        #departure day: simulate show-ups with binomial draws
        coach_show = int(rng.binomial(c, coach_show_up_prob))
        fc_show    = int(rng.binomial(f, first_class_show_up_prob))

        ob       = max(0, coach_show - coach_seats) #coach passengers without a seat
        fc_empty = max(0, first_class_seats - fc_show)
        bump_fc  = min(ob, fc_empty) #bump overbooked coach passengers to first class
        bump_off = ob - bump_fc #remaining overbooked passengers denied boarding

        cost = bump_fc * bump_to_fc_cost + bump_off * bump_off_plane_cost

        ob_costs[sim]   = cost
        overbooked[sim] = ob
        kicked[sim]     = bump_off
        profits[sim]    = disc_revenue - disc_t * cost #overbooking cost incurred at t=365

    return {
        'profits':    profits,
        'ob_costs':   ob_costs,
        'coach_sold': coach_sold,
        'fc_sold':    fc_sold,
        'overbooked': overbooked,
        'kicked':     kicked,
    }
```

```python
#choices for Policy A: hard-cap OB=9, same 6 price combos as Parts 1 and 2
choices_A = [
    (low_coach_price,  low_coach_prob,  low_first_class_price,  low_first_class_prob),
    (low_coach_price,  low_coach_prob,  high_first_class_price, high_first_class_prob),
```

```python
    (med_coach_price,  med_coach_prob,  low_first_class_price,  low_first_class_prob),
    (med_coach_price,  med_coach_prob,  high_first_class_price, high_first_class_prob),
    (high_coach_price, high_coach_prob, low_first_class_price,  low_first_class_prob),
    (high_coach_price, high_coach_prob, high_first_class_price, high_first_class_prob),
]

#choices for Policy B: flexible no-sale OB=30, adds no-sale option for coach
choices_B = choices_A + [
    (no_coach_price, no_coach_prob, low_first_class_price,  low_first_class_prob),
    (no_coach_price, no_coach_prob, high_first_class_price, high_first_class_prob),
]

#re-run both DPs to get fresh V and U matrices
choices = choices_A
V_A, U_A, profit_A_dp = simulate_policy(9) #Policy A: OB=9, hard cap
print(f"Policy A DP profit: ${profit_A_dp:,.2f}")

choices = choices_B
V_B, U_B, profit_B_dp = simulate_policy(30) #Policy B: OB=30, no-sale option
print(f"Policy B DP profit: ${profit_B_dp:,.2f}")

#run forward simulations with 10,000 runs each
print("\nSimulating Policy A (Hard Cap OB=9)...")
sim_A = forward_simulate(U_A, choices_A, max_coach_tix=coach_seats + 9,  n_sims=10_000)

print("Simulating Policy B (No-Sale, OB=30)...")
sim_B = forward_simulate(U_B, choices_B, max_coach_tix=coach_seats + 30, n_sims=10_000)

print("Done.")
```

```
Policy A DP profit: $42,171.35
Policy B DP profit: $42,177.14

Simulating Policy A (Hard Cap OB=9)...
Simulating Policy B (No-Sale, OB=30)...
Done.
```

Results

- Simulated Mean Profit (Policy A): **$42,183.65**

- Simulated Mean Profit (Policy B): **$42,102.30**

The table below summarizes the key statistics across both policies from the 10,000 simulations:

Metric	**Policy A (OB=9)**	**Policy B (OB=30, No-Sale)**
DP Expected Profit	$42,171.35	$42,177.14
Simulated Mean Profit	$42,183.65	$42,102.30
Simulated Std Dev	$906.11	$885.61
5th Percentile (downside)	$40,689.82	$40,613.15
95th Percentile (upside)	$43,664.46	$43,552.41
% Flights with Coach Overbooking	84.5%	86.5%
% Flights with Passengers Denied Boarding	73.0%	76.3%
Mean OB Cost (when > $0)	$1,230.61	$1,387.28
Mean Coach Tickets Sold	108.4	108.9
Mean FC Tickets Sold	19.6	19.5
Mean Overbooked Passengers	3.14	3.59

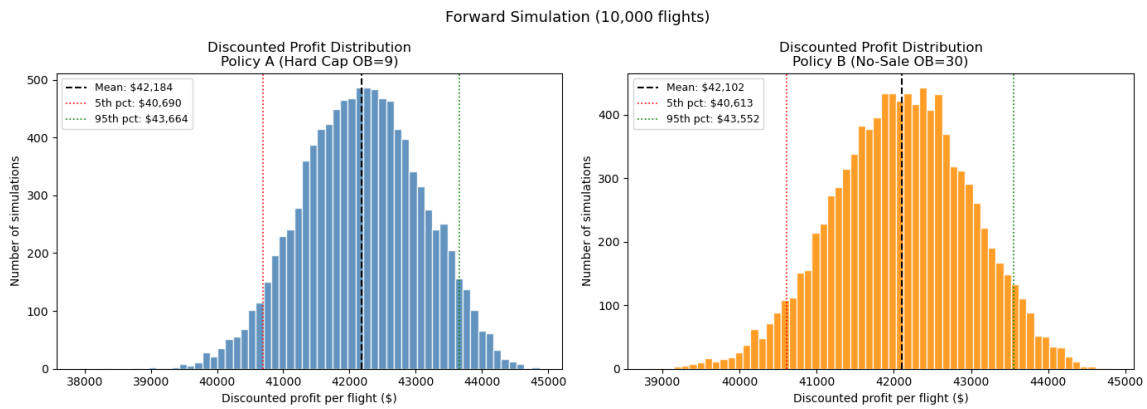

Figure 14. Profit Distributions - 10,000 flights (side-by-side histograms)

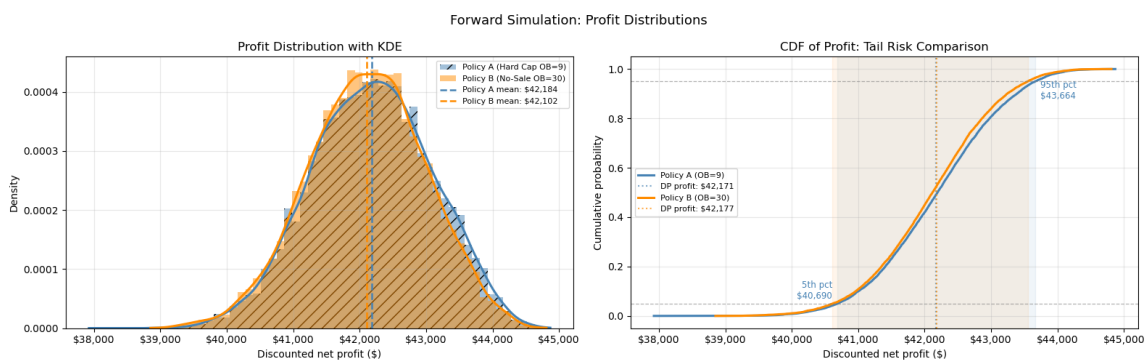

Figure 15. Profit Distributions (KDE overlay and the CDF plot)

The simulated means land close to their respective DP expected values, which confirms that our backward induction solution is correct and that the optimal policy actually achieves its theoretical profit in practice. Policy A's simulated mean ($42,183.65) sits slightly above its DP value, consistent with Monte Carlo noise. Policy B's simulated mean ($42,102.30) falls roughly $75 below its DP value, suggesting the no-sale option's theoretical value is harder to fully capture in practice.

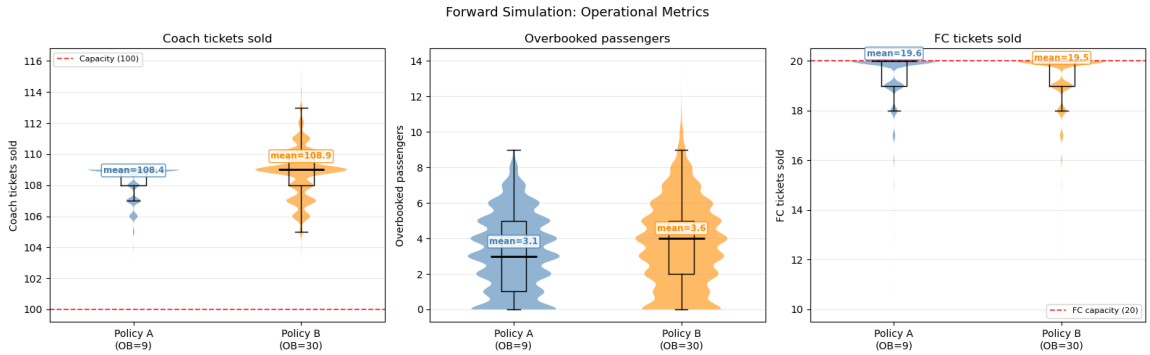

Figure 16. Operational Metrics (box plots showing coach tickets sold, overbooked passengers, and FC tickets sold)

The most important finding is that in forward simulation, Policy A outperforms Policy B by **$81.35** - the reverse of the DP ranking. The DP predicted a $5.79 advantage for Policy B; the simulation flips this. The no-sale flexibility that looked marginally beneficial in expectation comes at a real realized cost: withheld ticket revenue is certain, while the overbooking costs it was meant to prevent are probabilistic and often do not materialize. The dynamic program finds that selling a ticket is almost always better than withholding it.

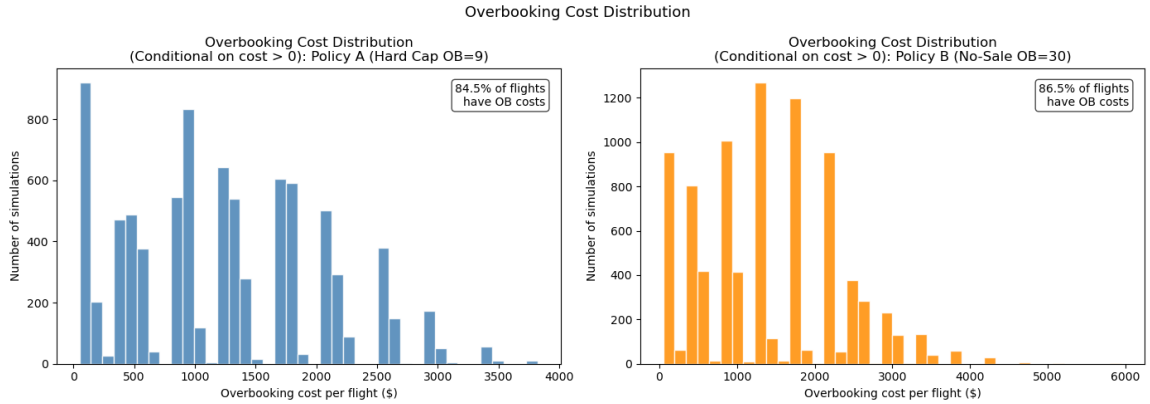

Figure 17. Overbooking Cost Distribution (conditional histograms of overbooking cost)

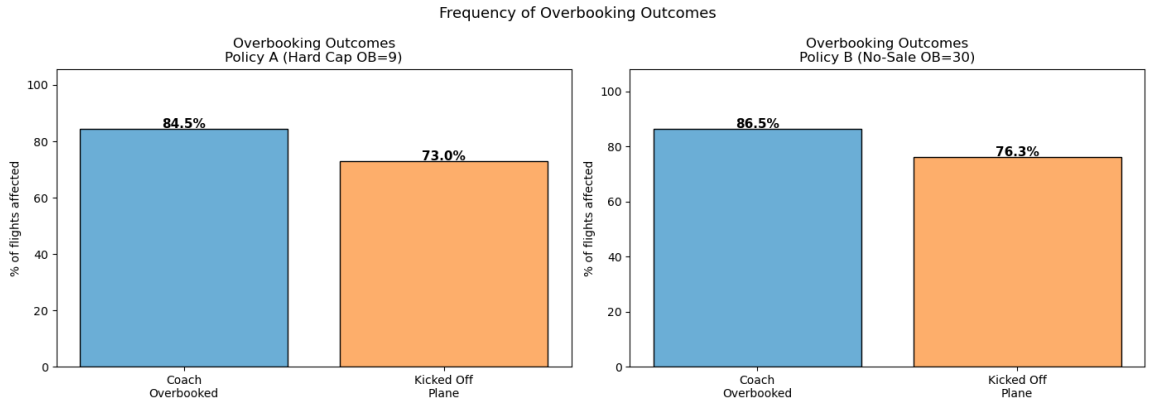

Figure 18: Frequency of Overbooking Outcomes (bar charts showing % of flights with coach overbooking vs. passengers kicked off)

While the simulated profits favor Policy A, Policy B retains real operational value: its no-sale option

acts as a safety valve near departure, withholding coach sales when many seats are already sold and overbooking costs would be high. This keeps the worst-case outcomes slightly less severe, which matters for an airline that faces reputational or regulatory costs from denied boardings.

The overbooking cost histograms show that most simulations with any overbooking cost cluster at relatively modest values, but there are rare tail events with costs exceeding $4,000 (Policy A) and $6,000+ (Policy B). Notably, overbooking is not a tail event in frequency, it occurs on 84.5% of Policy A flights and 86.5% of Policy B flights - but the *number* of affected passengers is right-skewed, with means of 3.14 and 3.59 respectively. Revenue uncertainty across the 365-selling window remains the dominant source of profit variability; overbooking policy cannot eliminate most of that volatility.

Policy recommendation: Policy A (Hard Cap OB=9) maximizes expected profit in both theory and simulation and is simpler to implement. Policy B is a reasonable choice only if the airline is particularly sensitive to worst-case denied boarding events, since the no-sale flexibility provides slight downside protection at a small cost in expected profit.

Conclusion

This project set out to determine the optimal daily pricing strategy and overbooking limit for a single flight over a 365-day selling window, using dynamic programming to maximize expected discounted profit. Across six parts, we progressively built a more realistic and complete model of the problem.

Starting with a 5-seat overbooking cap, we established the baseline framework: a backward induction algorithm that evaluates every possible state (day, coach tickets sold, first class tickets sold) and stores the optimal pricing decision at each one. The resulting policy was intuitive - charge high prices early when seats are scarce relative to time remaining, and drop prices only as departure approaches and unsold inventory becomes a liability. This yielded a baseline expected profit of $41,909.75.

By systematically searching overbooking limits from 5 to 20 extra seats, we found that 9 extra coach seats is the optimal cap, pushing expected profit to $42,171.35. The gain comes almost entirely from selling those 4 additional tickets at the same pricing structure, not from repricing. Beyond 9 seats, the growing risk of denied boardings and upgrade costs outweighs the revenue from additional ticket sales.

Adding a no-sale option in Part 3 - allowing the airline to withhold coach tickets on days when overbooking risk is too high raised expected profit only marginally to $42,177.14, a $5.79 gain over OB=9. The no-sale option is chosen in just 7% of states, almost exclusively when many coach tickets have already been sold. Its practical value is not in generating more revenue, but in giving the pricing team a lever to manage overbooking exposure in the most dangerous inventory states.

The sensitivity analysis in Part 4 revealed where estimation effort is best spent. First-class demand probabilities are by far the most influential inputs: a 1 percentage point error in the $500 first class sale probability costs approximately $151 in expected profit, while the same error in the $300 coach probability costs less than $0.50. This is a direct consequence of the pricing structure - high-priced first class tickets are sold early in the window, when the discount factor is close to 1, making each sale highly valuable. The $350 coach tier is the only coach parameter worth careful estimation, with a sensitivity of $112 per percentage point.

Introducing time-varying demand in Part 5 - where demand scales from 75% of baseline at Day 0 to 125% at departure confirmed that the optimal overbooking limit of 9 seats is robust to seasonal patterns. However, the demand structure itself hurts profitability: because revenue is discounted at 17% annually, the late-season surge in demand is worth less in present value than equivalent flat demand. The seasonal model produces $313.65 less in expected profit than the flat model at the same overbooking limit, and the policy shifts slightly, holding higher prices longer early in the window when demand is suppressed.

Finally, the forward simulation in Part 6 validated everything numerically across 10,000 runs. Both Policy A (OB=9) and Policy B (No-Sale, OB=30) achieved simulated mean profits close to their DP values, confirming the theoretical solution holds in practice. Notably, the simulation reversed the DP ranking: Policy A earned $42,183.65 on average versus $42,102.30 for Policy B, an $81.35 gap in favor of the simpler hard-cap policy. The no-sale option's theoretical edge does not fully survive realized randomness - withheld ticket revenue is certain, while the overbooking costs it was meant to prevent often do not materialize. The simulation also made clear that the dominant source of profit variability is demand uncertainty - how many tickets actually sell across 365 days, not overbooking policy. Overbooking itself is a near-certain occurrence, happening on over 84% of simulated flights, but the number of affected passengers is small on average and the associated costs are modest relative to total revenue.

Taken together, the analysis points to a clear practical recommendation: implement Policy A with a hard cap of 9 overbooked coach seats. It captures nearly all available expected profit, is simple to communicate and enforce, and performs well across the sensitivity and seasonality scenarios tested. The no-sale option (Policy B) is a reasonable addition if the airline faces significant reputational or regulatory costs for denied boardings, since it provides slight downside protection at virtually no cost in expected profit. In either case, the most valuable operational investment is in accurately estimating first-class demand, particularly the $500 ticket sale probability as errors there propagate directly and meaningfully into revenue.